

Python 编写规范（学生版）

基于 Python 官方 PEP8 风格指南整理

适用于：Python 初学者 / 学生 / 教学课堂 / 课程作业

为什么需要代码规范？

代码规范最重要的目标：

让别人容易读懂你的代码，也让未来的自己容易改代码。

规范不是限制，而是帮助：

- 提高代码可读性
 - 降低 Bug
 - 方便检查作业
 - 方便团队协作
 - 培养工程化思维
-

0. 总体原则

写 Python 代码时，先记住三句话：

1. 代码要能运行
2. 代码要容易读懂
3. 代码要方便修改

对于初学者，优先写清楚，不要过早追求“高级写法”和“一行代码解决所有问题”。

1. 基本格式规范

这是所有 Python 程序都要遵守的基础规范。

1.1 缩进使用 4 个空格

Python 使用缩进表示代码层级。

 正确

```
if score >= 60:  
    print("及格")
```

✗ 错误

```
if score >= 60:  
    print("及格")
```

不要混用 Tab 和空格，否则容易出现：

```
IndentationError
```

1.2 一行只做一件事

✓ 正确

```
name = "Tom"  
age = 18  
print(name)
```

✗ 错误

```
name = "Tom"; age = 18; print(name)
```

1.3 运算符两边加空格

常见运算符包括：=, +, -, *, /, //, %, ==, >=, <=。

✓ 正确

```
total = price * count  
average = total / 3
```

```
is_pass = score >= 60
```

✖ 错误

```
total=price*count  
average=total/3  
is_pass=score>=60
```

1.4 适当使用空行

空行用于分隔不同功能的代码，让代码更容易阅读。

✔ 推荐

```
name = input("请输入姓名: ")  
age = int(input("请输入年龄: "))  
  
print("姓名: ", name)  
print("年龄: ", age)
```

2. 命名规范

命名的目标是：

看到名字，就大概知道它表示什么。

2.1 变量名使用小写加下划线

变量名使用 `snake_case`：

- 全部小写
- 单词之间用下划线连接
- 不使用拼音缩写和无意义字母

✔ 正确

```
student_name = "Tom"
student_age = 18
total_score = 285
```

✗ 错误

```
studentName = "Tom"
xsxm = "Tom"
a = 18
```

如果变量只在很小范围内临时使用，`i`, `j`, `x`, `y` 可以接受。

2.2 函数名使用小写加下划线

✓ 正确

```
def get_average_score():
    pass

def print_student_info():
    pass
```

✗ 错误

```
def GetAverageScore():
    pass

def printstudentinfo():
    pass
```

2.3 类名使用大驼峰

类名使用 `PascalCase`:

- 每个单词首字母大写

- 不使用下划线

✓ 正确

```
class Student:
    pass

class StudentManager:
    pass
```

✗ 错误

```
class student:
    pass

class student_manager:
    pass
```

2.4 常量名使用全大写

常量表示程序中不希望被修改的固定值。

✓ 正确

```
MAX_SCORE = 100
PASS_SCORE = 60
DEFAULT_ENCODING = "utf-8"
```

✗ 错误

```
max_score = 100
passScore = 60
```

2.5 文件名使用小写加下划线

Python 文件名建议：

- 全小写
- 使用下划线连接单词
- 不使用空格

✓ 正确

```
student_manager.py  
score_statistics.py  
file_reader.py
```

✗ 错误

```
StudentManager.py  
score statistics.py  
文件读取.py
```

3. 字符串规范

字符串是学生最早接触的数据类型之一，建议从一开始就养成清晰写法。

3.1 字符串使用单引号或双引号都可以

同一个文件中尽量保持一致。

✓ 推荐

```
name = "Tom"  
message = "欢迎学习 Python"
```

3.2 字符串中包含引号时，选择更清楚的写法

✓ 正确

```
message = "I'm a student."  
title = '他说: "Python 很有趣。"'
```

3.3 推荐使用 f-string 格式化字符串

当字符串中需要加入变量时，推荐使用 f-string。

推荐

```
name = "Tom"  
age = 18  
  
print(f"{name} 今年 {age} 岁")
```

不推荐

```
print(name + " 今年 " + str(age) + " 岁")
```

3.4 字符串不要过长

一行字符串太长时，可以拆成多行。

推荐

```
message = (  
    "Python 是一门适合初学者学习的编程语言，"  
    "它语法清晰，应用场景广泛。"  
)
```

4. 基本数据类型规范

基本数据类型包括：整数、浮点数、布尔值和空值。

4.1 使用有意义的变量名

✅ 正确

```
student_count = 45  
average_score = 82.5  
is_pass = True
```

❌ 错误

```
a = 45  
b = 82.5  
c = True
```

4.2 布尔变量建议使用 is / has / can 开头

✅ 推荐

```
is_pass = score >= 60  
has_homework = True  
can_submit = False
```

4.3 判断布尔值时不要和 True / False 比较

✅ 正确

```
if is_pass:  
    print("通过")  
  
if not has_homework:  
    print("没有作业")
```

❌ 错误


```
if is_pass == True:
    print("通过")

if has_homework == False:
    print("没有作业")
```

4.4 和 None 比较时使用 is

✅ 正确

```
result = None

if result is None:
    print("暂无结果")
```

❌ 错误

```
if result == None:
    print("暂无结果")
```

5. 复合数据类型规范

复合数据类型包括：列表、元组、字典、集合。

5.1 列表名建议使用复数或能表示集合意义的名称

✅ 正确

```
scores = [89, 76, 92]
student_names = ["Tom", "Jerry", "Alice"]
```

❌ 错误

```
score = [89, 76, 92]
list1 = ["Tom", "Jerry", "Alice"]
```

5.2 列表、字典、集合较长时分多行书写

✅ 推荐

```
student_names = [
    "Tom",
    "Jerry",
    "Alice",
    "Bob",
]
```

```
student = {
    "name": "Tom",
    "age": 18,
    "score": 92,
}
```

5.3 字典的键名要表达清楚

✅ 正确

```
student = {
    "name": "Tom",
    "age": 18,
    "class_name": "人工智能1班",
}
```

❌ 错误

```
student = {
    "n": "Tom",
    "a": 18,
```

```
"c": "人工智能1班",  
}
```

5.4 不要直接修改正在遍历的列表

✗ 错误

```
scores = [45, 67, 38, 90]  
  
for score in scores:  
    if score < 60:  
        scores.remove(score)
```

✓ 推荐

```
scores = [45, 67, 38, 90]  
passed_scores = []  
  
for score in scores:  
    if score >= 60:  
        passed_scores.append(score)
```

6. 判断结构规范

判断结构包括：`if`, `elif`, `else`。

6.1 条件表达式要清晰

✓ 正确

```
if score >= 60:  
    print("及格")
```

✗ 错误

```
if score >= 60 == True:
    print("及格")
```

6.2 多分支判断按逻辑顺序排列

✅ 推荐

```
if score >= 90:
    level = "优秀"
elif score >= 80:
    level = "良好"
elif score >= 60:
    level = "及格"
else:
    level = "不及格"
```

6.3 不要写空的 if / else 分支

如果暂时没有实现，可以写 `pass`，但提交作业前应补全。

```
if score >= 60:
    pass
else:
    print("需要继续努力")
```

6.4 条件太复杂时，先拆成变量

✅ 推荐

```
is_valid_score = score >= 0 and score <= 100
is_excellent = score >= 90

if is_valid_score and is_excellent:
    print("成绩有效，等级优秀")
```

7. 循环结构规范

循环结构包括：`for` 和 `while`。

7.1 遍历列表时使用有意义的循环变量

✅ 正确

```
scores = [89, 76, 92]

for score in scores:
    print(score)
```

❌ 错误

```
scores = [89, 76, 92]

for x in scores:
    print(x)
```

在很短的数字循环中，`i` 可以接受。

```
for i in range(5):
    print(i)
```

7.2 while 循环要有明确的结束条件

✅ 正确

```
count = 1

while count <= 5:
    print(count)
    count += 1
```

❌ 错误

```
count = 1

while count <= 5:
    print(count)
```

7.3 break 和 continue 不要滥用

使用 `break` 和 `continue` 时，要确保循环逻辑仍然容易理解。

推荐

```
for score in scores:
    if score < 0 or score > 100:
        print("发现无效成绩")
        break
```

7.4 嵌套循环不要太深

初学阶段建议嵌套循环不超过两层。层数太多时，应考虑拆成函数。

8. 函数规范

函数用于把一段可以重复使用的代码封装起来。

8.1 一个函数只完成一件事

推荐

```
def calculate_average(scores):
    total = sum(scores)
    return total / len(scores)
```

不推荐

```
def process_scores(scores):  
    total = sum(scores)  
    average = total / len(scores)  
    print(scores)  
    print(average)  
    with open("scores.txt", "w", encoding="utf-8") as file:  
        file.write(str(average))
```

8.2 函数名用动词或动词短语

✅ 推荐

```
def get_max_score(scores):  
    return max(scores)  
  
def print_student_info(student):  
    print(student)
```

8.3 参数名要表达含义

✅ 正确

```
def calculate_total(price, count):  
    return price * count
```

❌ 错误

```
def calculate_total(a, b):  
    return a * b
```

8.4 函数之间空两行

✅ 正确

```
def add(a, b):  
    return a + b
```

```
def subtract(a, b):  
    return a - b
```

8.5 有返回值就用 return，不要只 print

如果函数的结果后面还要继续使用，应该 `return`。

✅ 推荐

```
def calculate_average(scores):  
    return sum(scores) / len(scores)  
  
average = calculate_average([80, 90, 100])  
print(average)
```

❌ 不推荐

```
def calculate_average(scores):  
    print(sum(scores) / len(scores))
```

9. 模块规范

模块就是一个 Python 文件。合理使用模块可以让代码更清晰。

9.1 import 放在文件顶部

import 顺序建议：

1. 标准库
2. 第三方库
3. 自己编写的模块

✅ 推荐

```
import os
import random

import requests

from student_manager import StudentManager
```

9.2 不要使用 import *

❌ 不推荐

```
from math import *
```

✅ 推荐

```
import math

print(math.sqrt(16))
```

9.3 程序入口使用 main 函数

当程序内容较多时，建议使用 `main()` 组织主流程。

✅ 推荐

```
def main():
    scores = [80, 90, 100]
    average = sum(scores) / len(scores)
    print(average)

if __name__ == "__main__":
    main()
```

10. 类和对象规范

类用于描述一类事物，对象是类创建出来的具体实例。

10.1 类名使用大驼峰

✅ 正确

```
class Student:
    pass

class CourseManager:
    pass
```

10.2 实例属性写在 init 中

✅ 推荐

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

10.3 方法的第一个参数写 self

✅ 正确

```
class Student:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        print(f"大家好，我叫 {self.name}")
```

10.4 list / dict / set 不要作为类属性保存可变数据

这是初学者学习类时最容易遇到的 Bug。

✗ 错误示例

```
class Teacher:
    courses = []
```

问题：所有 `Teacher` 对象会共享同一个列表。

```
class Teacher:
    courses = []

t1 = Teacher()
t2 = Teacher()

t1.courses.append("Python")
print(t2.courses)
```

输出：

```
["Python"]
```

✓ 正确写法

```
class Teacher:
    def __init__(self):
        self.courses = []
```

这样每个对象都有自己的 `courses`。

10.5 类中的方法之间空一行，类与类之间空两行

✓ 推荐

```
class Student:
    def __init__(self, name):
        self.name = name

    def introduce(self):
        print(self.name)

class Teacher:
    def __init__(self, name):
        self.name = name
```

11. 文件操作规范

文件操作容易因为路径、编码、关闭文件等问题出错，应从一开始写规范。

11.1 使用 with open 打开文件

`with open` 可以自动关闭文件。

推荐

```
with open("data.txt", "r", encoding="utf-8") as file:
    content = file.read()
```

不推荐

```
file = open("data.txt", "r")
content = file.read()
file.close()
```

11.2 明确写出 encoding

处理中文文件时，建议写出 `encoding="utf-8"`。

推荐

```
with open("students.txt", "w", encoding="utf-8") as file:
    file.write("张三,18\n")
```

11.3 文件路径使用有意义的变量

✅ 推荐

```
file_path = "students.txt"

with open(file_path, "r", encoding="utf-8") as file:
    content = file.read()
```

11.4 读文件时注意文件是否存在

初学阶段可以先使用简单判断。

```
import os

file_path = "students.txt"

if os.path.exists(file_path):
    with open(file_path, "r", encoding="utf-8") as file:
        content = file.read()
        print(content)
else:
    print("文件不存在")
```

12. 注释规范

注释不是把代码翻译成中文，而是说明代码背后的原因。

12.1 注释解释为什么，不解释显而易见的操作

❌ 不好的注释

```
# x 加 1
x += 1
```

代码本身已经能看懂。

✅ 好的注释

```
# 学号从 1 开始编号
student_id += 1
```

12.2 复杂逻辑前可以加简短注释

```
# 先过滤无效成绩，再计算平均分
valid_scores = []

for score in scores:
    if 0 <= score <= 100:
        valid_scores.append(score)
```

12.3 不要保留大量无用代码

提交作业前，应删除不用的调试代码。

❌ 不推荐

```
# print(score)
# print(total)
# print(average)
```

13. 输入输出规范

课堂练习经常需要输入输出，建议保持提示清楚、输出整洁。

13.1 input 提示语要明确

✓ 推荐

```
name = input("请输入姓名：")  
age = int(input("请输入年龄："))
```

✗ 不推荐

```
name = input()  
age = int(input())
```

13.2 输出结果要有说明

✓ 推荐

```
print(f"平均分：{average}")
```

✗ 不推荐

```
print(average)
```

14. 常见错误提醒

14.1 混用 Tab 和空格

会导致：

```
IndentationError
```

14.2 命名随意

✗ 不推荐

```
a = 1
b = 2
c = a + b
```

✓ 推荐

```
price = 1
count = 2
total_price = price * count
```

14.3 一个函数太长

建议：

- 一个函数只做一件事
- 函数长度尽量控制在一屏以内
- 复杂功能拆成多个小函数

14.4 可变对象作为类属性

不要把列表、字典、集合作为类属性来保存每个对象自己的数据。

✗ 不推荐

```
class Student:
    scores = []
```

✓ 推荐

```
class Student:
    def __init__(self):
        self.scores = []
```


14.5 忘记处理数据类型转换

`input()` 得到的是字符串，需要计算时要转换类型。

✓ 正确

```
age = int(input("请输入年龄: "))
```

✗ 错误

```
age = input("请输入年龄: ")  
print(age + 1)
```

15. 推荐工具

现代 Python 开发通常会配合自动工具。初学阶段可以先了解，后续项目中再逐步使用。

Black

Black 可以自动格式化代码。

安装

```
pip install black
```

使用

```
black .
```

它会自动：

- 统一空格
- 自动换行
- 调整代码格式

Ruff

Ruff 是一个速度很快的 Python 检查工具。

安装

```
pip install ruff
```

使用

```
ruff check .
```

它可以检查：

- 命名问题
- 未使用变量
- import 顺序
- 潜在 Bug
- 常见 PEP8 问题

16. 课堂作业提交前检查清单

提交代码前，建议按下面清单检查一遍：

- 缩进是否统一使用 4 个空格
 - 变量名、函数名是否清楚
 - 运算符两边是否有空格
 - 是否一行只写一条语句
 - 字符串输出是否清楚
 - 判断条件是否容易理解
 - 循环是否有明确结束条件
 - 函数是否只做一件事
 - import 是否放在文件顶部
 - 文件操作是否使用 `with open`
 - 中文文件是否写了 `encoding="utf-8"`
 - 是否删除了无用的调试代码
-

最终总结

记住最重要的几条：

1. 使用 4 个空格缩进
2. 变量名和函数名使用小写加下划线
3. 类名使用大驼峰
4. 运算符两边加空格
5. 一行只做一件事
6. 判断布尔值不要写 `== True`
7. 函数尽量只完成一件事
8. `import` 放在文件顶部
9. 文件操作使用 `with open`
10. `list` / `dict` / `set` 不要作为实例数据的类属性

一句话总结

好代码的第一标准：

容易读懂。